As you'll see later, functions registered as beans are enhanced with extra features by the Spring Cloud Function framework. The beauty of this is that the business logic itself is not aware of the surrounding framework. You can evolve it independently and test it without being concerned about framework-related issues.

USING IMPERATIVE AND REACTIVE FUNCTIONS

Spring Cloud Function supports both imperative and reactive code, so you're free to implement functions using reactive APIs like `Mono` and `Flux`. You can also mix and match. For the sake of the example, let's implement the *label* function using Project Reactor. The input of the function will be the identifier of an order that has been packed, represented as a `Long` object. The output of the function will be the identifier of the order that has been labeled, resulting in the dispatching process being complete. We can model such data via a simple DTO, just like we did for `OrderAcceptedMessage`.

In the `com.polarbookshop.dispatcherservice` package, create an `OrderDispatchedMessage` record to hold the identifier for a dispatched order.

Listing 10.7 A DTO representing the event about orders being dispatched

```
package com.polarbookshop.dispatcherservice;

public record OrderDispatchedMessage (      ❶
  Long orderId
){}
```

❶ DTO containing the order identifier as a Long field